Fixed-Space Replacement

Peter J. Denning

© Copyright 2022, Peter J. Denning

Performance a Big Deal

- VM elegantly solves the mapping problem
- But it leaves us with a new problem: managing the up and down moves
- Originally called “overlay problem”, now “replacement problem”
- Memory gap very expensive
  - ratio DISK access time to RAM access time, typically 10^6
- Imperative to minimize page faults
  - Cannot avoid initial load fault for each used page of address space
  - Try to avoid as many re-loads as possible
- Replacement policy the deciding factor

© Copyright 2022, Peter J. Denning

Why Focus on Replacement?

- Almost all pages are reused repeatedly
- Replacing a page implies future page fault to recover it
- Choice of page to replace therefore influences amount of paging

© Copyright 2022, Peter J. Denning

History

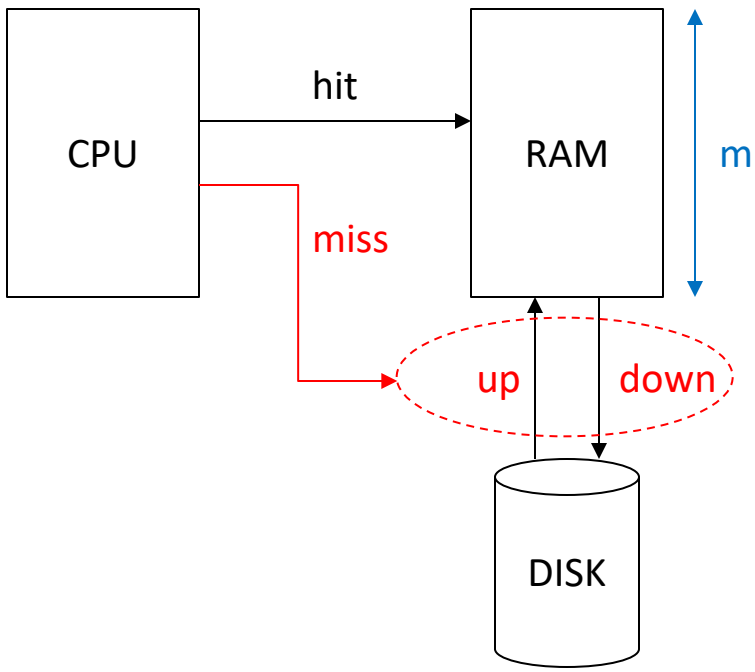
- First virtual memory on Atlas OS at Manchester, UK, 1960
- Its RAM was small and OS ran only one process at a time
  - Paging didn't have to be zero, just comparable with what programmers achieved with manual overlays
- They called their replacement algorithm "learning algorithm"
  - Monitoring use bits, computed a loop period for each page
  - Predicted next use of each page, selected the one farthest in future

© Copyright 2022, Peter J. Denning

- Learning algorithm did not work very well on other VMs
  - Different workloads
  - Not all pages were in tight loops
  - Some pages in multiple loops with different periods
- Massive search for “best” paging algorithm 1962-1970
- Discouraging conclusions
  - LRU (least recently used) – usually the best but high overhead
  - FIFO (first in first out) – usually the worst but low overhead
  - MIN (optimal) – consistently much better than LRU but not realizable

© Copyright 2022, Peter J. Denning

Singleton Memory Principle



System is seen as a single CPU accessing a single RAM of fixed size m pages. Mapping hits go direct to RAM. Misses trigger page faults that cause up and down page moves. The objective is to minimize up moves.

In a system with multiprogramming, the RAM here is a fixed region of the full RAM.

The choice of m and content of RAM is determined by the virtual memory manager.

© Copyright 2022, Peter J. Denning

- With the singleton memory view, designers do not need to worry about other processes sharing the RAM.
- Performance is based entirely on m, the size of the singleton RAM, and on the replacement policy. The performance measure is F(m), the total number of page faults during process execution.
- Much easier to think about than full multiprogramming with many interactions among processes.

© Copyright 2022, Peter J. Denning

Finding a good Replacement Policy

- Imagine an “evil” policy that aims to maximize page faults.
  - Always choose replacement to ensure next accessed page is missing
  - Gives a worst possible paging rate, 1 fault per access
- Now image a “benevolent” policy that aims to minimize page faults.
  - At every fault, replace the page that will not be reused for the longest time into the future
  - Maximizes the spacing between faults and therefore minimizes the total number of faults
  - Named MIN by Les Belady in 1966 – optimal in principle

© Copyright 2022, Peter J. Denning

- Both evil and benevolent polices can't be built because they rely on knowledge of the future

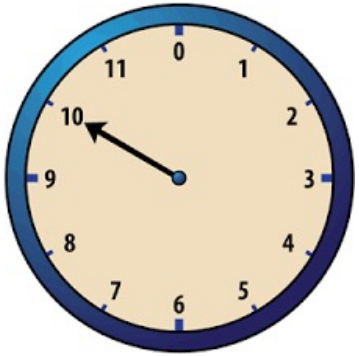
- What can be built? Many have been proposed but two have stood out for their use in operating systems:

- FIFO – memory a FIFO queue – replace the page in memory longest
  - No use bits, extremely simple (near-zero overhead) implementation

- LRU – replace the least recently used page in memory
  - Maintains an "order of recency" list to find the LRU
  - Consistently good performance, but high overhead implementation

© Copyright 2022, Peter J. Denning

CLOCK



- A hybrid combining the efficiency of FIFO with the effectiveness of LRU: FIFO with usage bits
- Originally called FINUFO, First In Not Used First Out (1968)
- Same as FIFO, except skip used pages while resetting their use bits
- Worst case: one complete cycle, shuts off all use bits
- Attraction: almost as simple as FIFO; protects used pages

© Copyright 2022, Peter J. Denning

Simulations of Policies

- To find F(m), the fault count function of a replacement policy, we can simulate the policy on an address trace for m=1,…,M
- Address trace is the sequence of page numbers in the successive memory accesses of the process.
- Time is measured in discrete steps, counting the number of memory accesses so far.
- Parameters: N = length of trace, M = number of distinct pages
- Notation: r(t) = page referenced at time t, t=1,2,…,N

© Copyright 2022, Peter J. Denning

For our example simulations, we use this address trace:

1 2 3 4 1 2 5 1 2 3 4 5

Here, N = 12 and M = 5

Note F(1) = N because no successive references repeat

And F(M) = M because first faults cannot be avoided

© Copyright 2022, Peter J. Denning

FIFO

- FIFO = first in first out
- m loaded pages in a circular list
- Index i tells which page is next for replacement
- On page fault, replace that page and set $i=(i+1) \bmod m$
- Pro: very simple implementation
- Con: consistently more faults than LRU

© Copyright 2022, Peter J. Denning

FIFO Example

r(t):	1	2	3	4	1	2	5	1	2	3	4	5	
m=2:	1	2	3	4	1	2	5	1	2	3	4	5	
		1	2	3	4	1	2	5	1	2	3	4	
faults:	x	x	x	x	x	x	x	x	x	x	x	x	F(2)=12

r(t):	1	2	3	4	1	2	5	1	2	3	4	5	
	1	2	3	4	1	2	5	5	5	3	4	4	
m=3:		1	2	3	4	1	2	2	2	5	3	3	
			1	2	3	4	1	1	1	2	5	5	
faults:	x	x	x	x	x	x	x			x	x		F(3)=9

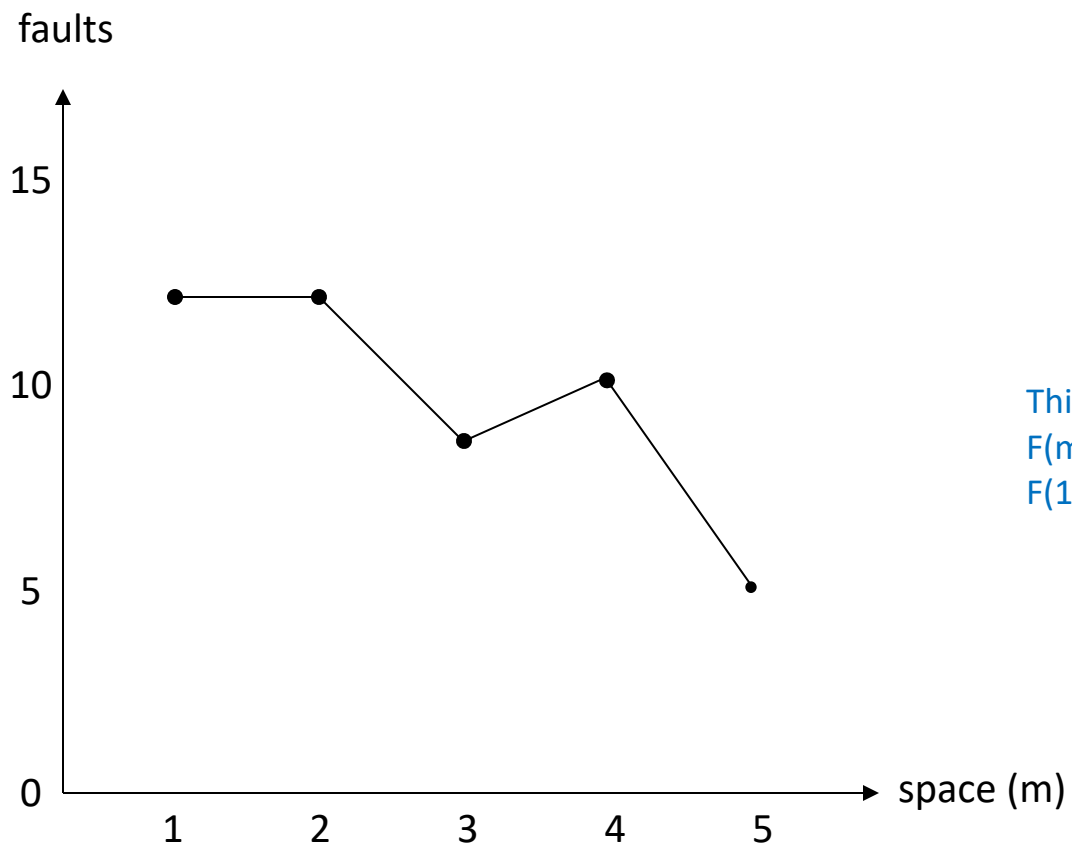
r(t):	1	2	3	4	1	2	5	1	2	3	4	5	
	1	2	3	4	4	4	5	1	2	3	4	5	
m=4:		1	2	3	3	3	4	5	1	2	3	4	
			1	2	2	2	3	4	5	1	2	3	
				1	1	1	2	3	4	5	1	2	
faults:	x	x	x	x			x	x	x	x	x	x	F(4)=10

Top row is address trace. Bottom row marks times of faults. Center columns are memory contents, arranged in FIFO order from top to bottom.

Fault occurs when page accessed is not in the immediately preceding memory contents – that is, r(t) is not in memory established at t-1

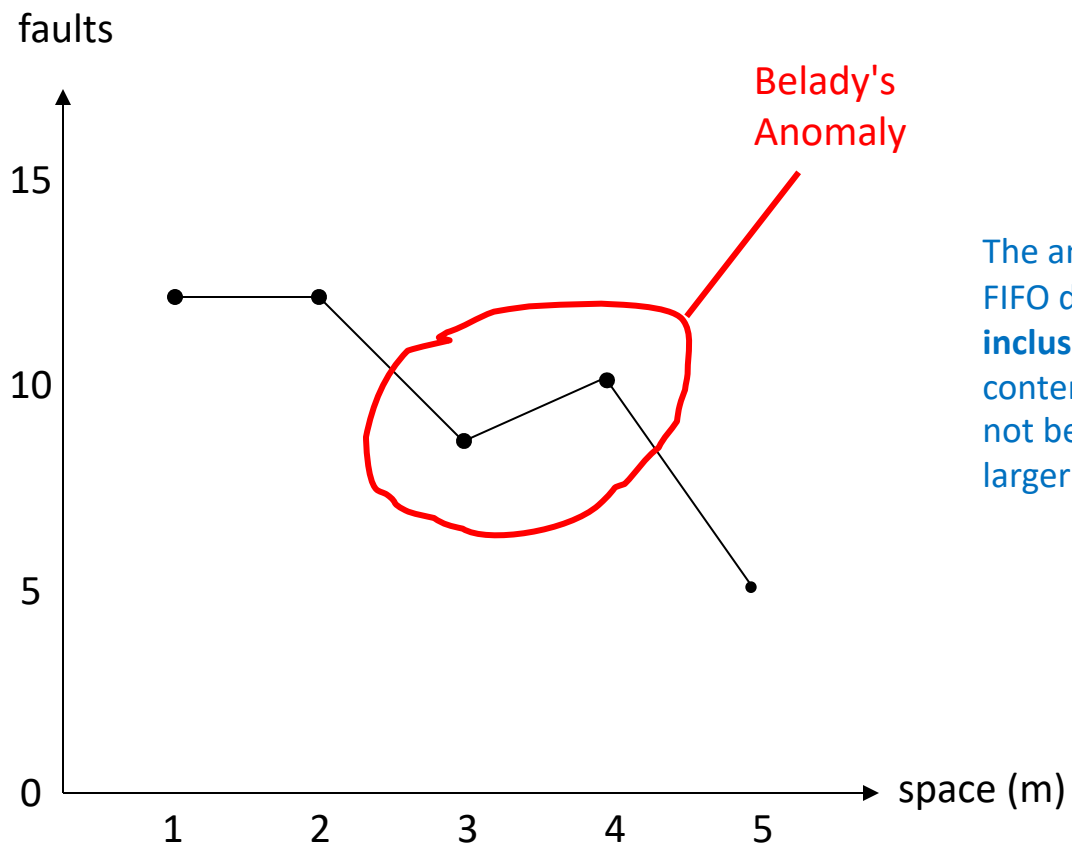
The highlighted memory contents show that adding a page of memory may not augment the old FIFO memory contents. The next reference, r(8)=1 causes a fault for m=4 but not for m=3.

© Copyright 2022, Peter J. Denning



This is a plot of FIFO's F(m). Note that F(1)=12 and F(5)=5

© Copyright 2022, Peter J. Denning



The anomaly occurs because FIFO does not satisfy the **inclusion property**: memory contents for smaller m may not be subsets of those for larger m (see example).

© Copyright 2022, Peter J. Denning

LRU

- LRU = least recently used
- Maintain vector called “LRU stack” ordering all M pages from most to least recently referenced
- Page to be replaced is at position m in the stack
- Pro: good performance, no anomalies
- Con: high overhead of constantly updating the stack

© Copyright 2022, Peter J. Denning

LRU Example

r(t):	1	2	3	4	1	2	5	1	2	3	4	5
m=2:	1	2	3	4	1	2	5	1	2	3	4	5
		1	2	3	4	1	2	5	1	2	3	4
faults:	x	x	x	x	x	x	x	x	x	x	x	x

F(2)=12

r(t):	1	2	3	4	1	2	5	1	2	3	4	5
	1	2	3	4	1	2	5	1	2	3	4	5
m=3:		1	2	3	4	1	2	5	1	2	3	4
			1	2	3	4	1	2	5	1	2	3
faults:	x	x	x	x	x	x	x			x	x	x

F(3)=10

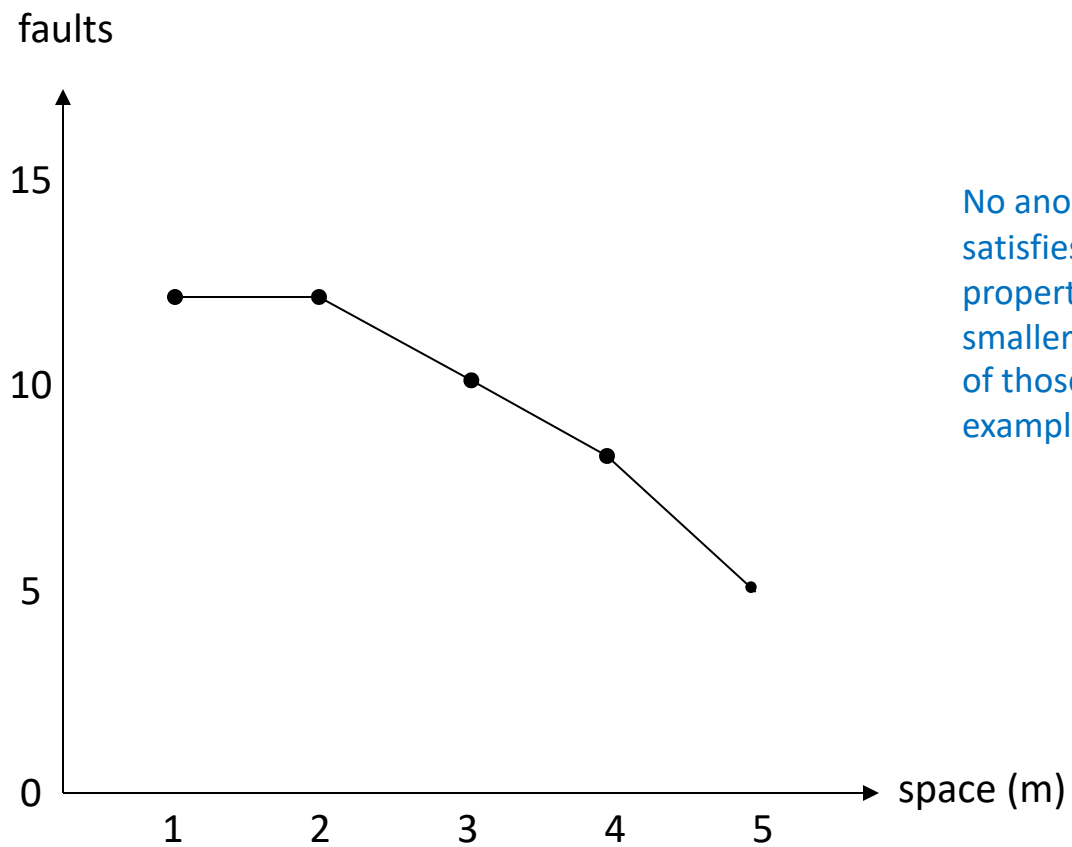
r(t):	1	2	3	4	1	2	5	1	2	3	4	5
	1	2	3	4	1	2	5	1	2	3	4	5
m=4:		1	2	3	4	1	2	5	1	2	3	4
			1	2	3	4	1	2	5	1	2	3
				1	2	3	4	4	4	5	1	2
faults:	x	x	x	x			x			x	x	x

F(4)=8

Top row is address trace. Bottom row marks times of faults. Center columns are memory contents, arranged in LRU order from top to bottom.

All LRU stacks satisfy the inclusion property: adding memory preserves the old contents and adds another page. That is because the m most recently used pages always include the m-1 most recently used.

© Copyright 2022, Peter J. Denning



No anomaly because LRU satisfies the inclusion property: memory contents for smaller m are subsets of those for larger m (see example).

© Copyright 2022, Peter J. Denning

MIN

- MIN = minimum possible faults (optimum)
- Replace page with longest forward distance until next reuse
- Attraction: benchmark for realizable policies
- Con: not realizable
- Can be used as post mortem benchmark to compare with LRU and FIFO – usually significantly better than LRU

© Copyright 2022, Peter J. Denning

MIN Example

r(t):	1	2	3	4	1	2	5	1	2	3	4	5	
m=2:	1	2	3	4	1	2	5	1	2	3	4	5	
		1	1	1	4	1	1	5	5	5	5	4	
faults:	x	x	x	x		x	x		x	x	x		F(2)=9

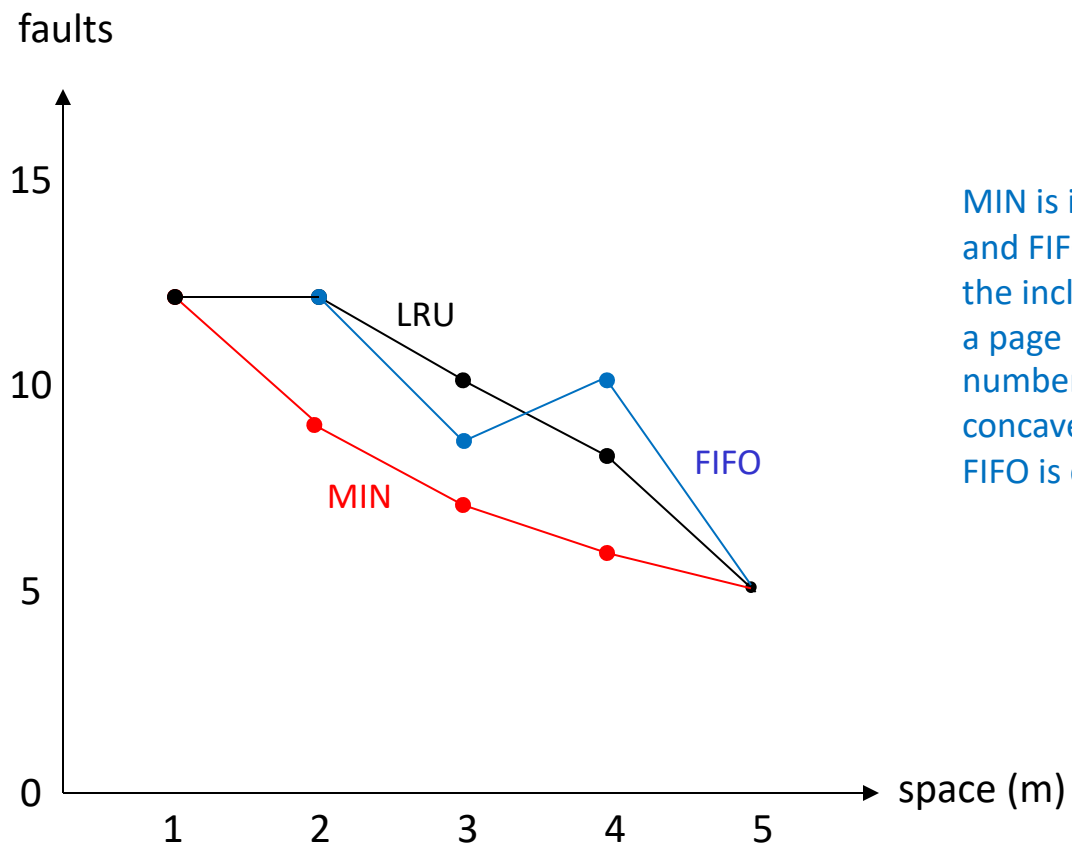
r(t):	1	2	3	4	1	2	5	1	2	3	4	5	
	1	2	3	4	1	2	5	1	2	3	4	5	
m=3:		1	1	1	4	1	1	5	5	5	5	4	
			2	2	2	4	2	2	1	1	1	1	
faults:	x	x	x	x			x			x	x		F(3)=7

r(t):	1	2	3	4	1	2	5	1	2	3	4	5	
	1	2	3	4	1	2	5	1	2	3	4	5	
m=4:		1	1	1	4	1	1	5	5	5	5	4	
			2	2	2	4	2	2	1	1	1	1	
				3	3	3	3	3	3	2	2	2	
faults:	x	x	x	x			x				x		F(4)=6

Top row is address trace. Bottom row marks times of faults. Center columns are memory contents. Order is determined by successive comparisons of page pushed down when referenced page moves up. If the page moving down is closer in the future, it settles in that position and the page there moves down. After last reference priority is determined by page number.

Note that MIN stacks satisfy the inclusion property.

© Copyright 2022, Peter J. Denning



MIN is indeed superior to LRU and FIFO. LRU and MIN satisfy the inclusion property -- adding a page never increases the number of faults. MIN is also concave up, but neither LRU nor FIFO is concave.

© Copyright 2022, Peter J. Denning